LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 3 & 4 – FUNGSI & PROSEDUR

NAMA : MOH. CHANDRA WARDANA

NIM : 109082530025

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori Modul 3

Modul ini membahas tentang fungsi dalam pemrograman, yaitu sekumpulan instruksi yang digunakan untuk menghasilkan suatu nilai dari input tertentu. Fungsi selalu mengembalikan nilai dengan menggunakan *return*, sehingga berbeda dengan prosedur. Dalam praktiknya, fungsi sering digunakan untuk menghitung suatu nilai, dimasukkan ke dalam variabel, atau dijadikan bagian dari perhitungan lain. Karena itu, penamaan fungsi biasanya dibuat sesuai dengan tujuan hasilnya agar lebih mudah dipahami.

Selanjutnya dijelaskan bagaimana cara membuat atau mendeklarasikan fungsi, baik dalam bentuk algoritma maupun bahasa pemrograman seperti Go. Struktur fungsi terdiri dari nama fungsi, parameter, tipe data hasil, dan bagian isi yang berisi proses perhitungan. Contohnya adalah fungsi untuk menghitung volume tabung, di mana nilai jari-jari dan tinggi diproses menjadi hasil volume. Dari contoh ini terlihat bahwa fungsi membantu membuat program lebih rapi dan tidak berulang-ulang menulis kode yang sama.

Terakhir, modul ini juga menjelaskan cara penggunaan fungsi dalam program. Fungsi bisa dipanggil dan hasilnya langsung disimpan ke variabel, digabung dalam operasi lain, atau langsung ditampilkan. Bahkan, satu fungsi bisa memanggil fungsi lain, seperti pada contoh faktorial dan permutasi. Dengan cara ini, program jadi lebih terstruktur dan lebih mudah dikembangkan untuk menyelesaikan masalah yang lebih kompleks.

B. Guided

1. Hitung Luas Persegi Panjang dengan Function

```go
package main

import "fmt"

func hitungLuasPersegiPanjang(panjang int, lebar int) int {
    luas := panjang * lebar
    return luas
}

func main() {
    p := 10
    l := 5

    hasil := hitungLuasPersegiPanjang(p, l)

    fmt.Printf("Luas persegi panjang dengan panjang %d dan lebar %d adalah: %d\n", p, l, hasil)
}
```

Penjelasan :

Kode tersebut merupakan program bahasa Go (Golang) yang dirancang untuk menghitung luas persegi panjang secara terstruktur. Program dimulai dengan deklarasi package main dan impor package fmt untuk keperluan input-output, kemudian mendefinisikan sebuah fungsi bernama hitungLuasPersegiPanjang yang menerima dua parameter integer (panjang dan lebar) serta mengembalikan hasil perkalian keduanya. Di dalam fungsi utama atau main, variabel p dan l diinisialisasi dengan nilai masing-masing 10 dan 5 menggunakan operator deklarasi singkat :=, yang kemudian nilainya dikirim ke fungsi penghitung luas untuk diproses. Hasil akhir dari fungsi tersebut disimpan dalam variabel hasil dan ditampilkan ke layar menggunakan fmt.Printf, di mana format %d digunakan sebagai penampung nilai angka sehingga menghasilkan kalimat "Luas persegi panjang dengan panjang 10 dan lebar 5 adalah: 50".

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> cd "Week 3 - Modul 3/Guided/guided1"
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Guided\guided1> go run guided1.go
Luas persegi panjang dengan panjang 10 dan lebar 5 adalah: 50
```

2. Cek Bilangan Genap Dengan Function

```go
package main

import "fmt"

func cekGenap(angka int) bool {
    if angka%2 == 0 {
        return true
    }
    return false
}

func main() {
    angkaTes := 9

    hasilGenap := cekGenap(angkaTes)

    fmt.Printf("Apakah %d itu bilangan genap? Jawabannya: %t\n", angkaTes, hasilGenap)
}
```

Penjelasan Code :

Program ini bertujuan untuk mengecek apakah suatu bilangan termasuk genap atau tidak dengan menggunakan fungsi. Fungsi cekGenap menerima satu parameter berupa angka bertipe integer, kemudian melakukan pengecekan menggunakan operator modulus (%) untuk melihat apakah angka tersebut habis dibagi 2. Jika hasilnya 0, maka fungsi mengembalikan nilai true (genap), jika tidak maka false (ganjil). Pada fungsi main, sebuah variabel angkaTes diisi dengan nilai 9, lalu dikirim ke fungsi cekGenap dan hasilnya disimpan dalam variabel hasilGenap. Terakhir, program menampilkan hasil pengecekan ke layar menggunakan fmt.Printf, dengan format yang menyertakan nilai angka dan hasil boolean (true/false), sehingga pengguna bisa langsung mengetahui apakah angka tersebut genap atau tidak.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> cd "Week 3 - Modul 3/Guided/guided2"
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Guided\guided2> go run guide
d2.go
Apakah 9 itu bilangan genap? Jawabannya: false
```

C. Unguided

1. Soal Unguided 1
   Kombinasi & Permutasi

```go
package main

import "fmt"

func factorial(n int) int {
    var hasil int = 1
    for i := 1; i <= n; i++ {
        hasil *= i
    }
    return hasil
}

func permutasi(n, r int) int {
    return factorial(n) / factorial(n-r)
}

func kombinasi(n, r int) int {
    return factorial(n) / (factorial(r) * factorial(n-r))
}

func main() {
    var a, b, c, d int
    fmt.Print("Masukkan nilai a: ")
    fmt.Scan(&a)
    fmt.Print("Masukkan nilai b: ")
    fmt.Scan(&b)
    fmt.Print("Masukkan nilai c: ")
    fmt.Scan(&c)
    fmt.Print("Masukkan nilai d: ")
    fmt.Scan(&d)

    fmt.Printf("hasil permutasi %d dan %d adalah: %d\n", a, c, permutasi(a, c))
    fmt.Printf("hasil kombinasi %d dan %d adalah: %d\n", a, c, kombinasi(a, c))
    fmt.Printf("hasil permutasi %d dan %d adalah: %d\n", b, d, permutasi(b, d))
    fmt.Printf("hasil kombinasi %d dan %d adalah: %d\n", b, d, kombinasi(b, d))
}
```

Penjelasan :
Kode di atas disusun dengan membuat sebuah fungsi dasar bernama factorial yang menggunakan perulangan for untuk menghitung hasil perkalian beruntun dari angka satu hingga n. Fungsi ini kemudian dimanfaatkan oleh fungsi permutasi dan kombinasi untuk mengimplementasikan rumus matematika yang diberikan, di mana permutasi membagi faktorial n dengan faktorial selisih n dan r, sementara kombinasi menambahkan pembagi berupa faktorial r. Di dalam fungsi utama, program menerima empat input bilangan asli dan memanggil kedua fungsi tersebut secara bergantian untuk mencetak hasil perhitungan sesuai dengan format ekspektasi output yang diminta.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Unguided\unguided1.go> go ru
n unguided-no1.go
Masukkan nilai a: 5
Masukkan nilai b: 10
Masukkan nilai c: 3
Masukkan nilai d: 10
hasil permutasi 5 dan 3 adalah: 60
hasil kombinasi 5 dan 3 adalah: 10
hasil permutasi 10 dan 10 adalah: 3628800
hasil kombinasi 10 dan 10 adalah: 1
```

2. Soal Unguided 2
   Komposisi Fungsi

```go
package main

import "fmt"

func f(x int) int {
    return x * x
}

func g(x int) int {
    return x - 2
}

func h(x int) int {
    return x + 1
}

func main() {
    var a, b, c int
    fmt.Print("Masukkan nilai x: ")
    fmt.Scan(&a)
    fmt.Print("Masukkan nilai y: ")
    fmt.Scan(&b)
    fmt.Print("Masukkan nilai z: ")
    fmt.Scan(&c)

    fmt.Printf("F(G(H( %d ))) : %d\n", a, f(g(h(a))))
    fmt.Printf("G(H(F( %d ))) : %d\n", b, g(h(f(b))))
    fmt.Printf("H(F(G( %d ))) : %d\n", c, h(f(g(c))))
}
```

Penjelasan :

Struktur kode ini didasarkan pada definisi tiga fungsi matematika sederhana yang masing-masing mengembalikan nilai kuadrat, pengurangan, dan penjumlahan sesuai dengan instruksi soal. Inti dari program ini adalah penerapan konsep fungsi komposisi di mana hasil keluaran dari satu fungsi langsung dijadikan masukan bagi fungsi lainnya, seperti pada perintah f(g(h(a))) yang berarti nilai a diproses terlebih dahulu oleh fungsi h, kemudian hasilnya diproses oleh g, dan terakhir oleh f. Program kemudian menerima tiga input berbeda dan menjalankan urutan komposisi yang bervariasi untuk setiap variabel tersebut guna menghasilkan output yang sesuai dengan tampilan pada terminal yang disediakan.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Unguided\unguided2.go> go ru
n unguided-no2.go
Masukkan nilai x: 7
Masukkan nilai y: 2
Masukkan nilai z: 10
F(G(H( 7 ))) : 36
G(H(F( 2 ))) : 3
H(F(G( 10 ))) : 65
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Unguided\unguided2.go> go ru
n unguided-no2.go
Masukkan nilai x: 15
Masukkan nilai y: 8
Masukkan nilai z: 3
F(G(H( 15 ))) : 196
G(H(F( 8 ))) : 63
H(F(G( 3 ))) : 2
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Unguided\unguided2.go> go ru
n unguided-no2.go
Masukkan nilai x: 5
Masukkan nilai y: 17
Masukkan nilai z: 8
F(G(H( 5 ))) : 16
G(H(F( 17 ))) : 288
H(F(G( 8 ))) : 37
PS D:\109082530025_MOH-Chandra-Wardana_\Week 3 - Modul 3\Unguided\unguided2.go> go ru
n unguided-no2.go
```

Kesimpulan :

Fungsi merupakan subprogram berisi rangkaian instruksi yang dirancang untuk memetakan input menjadi suatu output tertentu dengan karakteristik utama wajib memiliki deklarasi tipe data pengembalian dan menggunakan kata kunci return di dalam badan algoritmanya. Berbeda dengan prosedur, fungsi dapat digunakan secara fleksibel sebagai bagian dari ekspresi, argumen dalam subprogram lain, atau langsung di-assign ke dalam suatu variabel karena sifatnya yang selalu menghasilkan nilai. Implementasi praktisnya meliputi perhitungan matematis yang kompleks seperti operasi faktorial, permutasi, dan kombinasi, hingga pengolahan fungsi komposisi serta penentuan posisi titik dalam geometri lingkaran. Melalui penggunaan fungsi, struktur program menjadi lebih modular dan efisien, terutama ketika sebuah perhitungan perlu dipanggil berulang kali dalam logika pemrograman, baik dalam notasi algoritma maupun bahasa Go.

D. Dasar Teori Modul 4

Modul ini menjelaskan tentang prosedur dalam pemrograman, yang sebenarnya mirip seperti fungsi tapi tanpa nilai kembalian. Jadi kalau fungsi itu pasti ada *return*, prosedur justru fokus ke proses atau aksi yang dilakukan, misalnya menampilkan output atau mengubah nilai tertentu di program. Karena itu, penamaan prosedur biasanya pakai kata kerja seperti *cetak*, *hitung*, atau *tampilkan*, biar jelas kalau dia menjalankan suatu tindakan, bukan menghasilkan nilai.

Di bagian berikutnya dijelaskan cara penulisan prosedur, baik dalam pseudocode maupun Go. Strukturnya cukup sederhana, ada nama prosedur, parameter, dan isi prosedur yang berisi langkah-langkahnya. Contoh yang diberikan seperti prosedur untuk mencetak deret Fibonacci, jadi prosedur itu langsung bekerja menampilkan hasil tanpa perlu mengembalikan nilai. Dari sini kelihatan kalau prosedur cocok dipakai buat hal-hal yang sifatnya “melakukan sesuatu” di program, bukan menghitung hasil untuk dipakai lagi.

Terakhir, modul ini juga bahas cara pemanggilan prosedur dan konsep parameter. Prosedur dipanggil cukup dengan menuliskan nama dan argumennya, bisa dari variabel atau langsung nilai. Yang menarik, ada pembahasan tentang *pass by value* dan *pass by reference*, di mana kalau pakai reference, prosedur bisa mengubah nilai dari variabel asli di program utama. Jadi walaupun prosedur tidak punya return, dia tetap bisa “mengirim

balik” hasil lewat parameter. Ini yang bikin prosedur tetap fleksibel dan sering dipakai dalam program yang lebih kompleks.

E. Guided 3

3. Perbedaan Mekanisme Pass by Value dan Pass by Reference dalam Bahasa Go

```go
package main

import "fmt"

func tambahValue(x int) {
    x = x + 10
    fmt.Println("Nilai x didalam prosdur tambahValue (pass by value) : ", x)
}

func tambahReference(x *int) {
    *x = *x + 10
    fmt.Println("Nilai x didalam prosdur tambahReference (pass by refeence) : ", *x)
}

func main() {
    var y int = 5

    fmt.Println("nilai y awal : ", y)
    tambahValue(y)
    fmt.Println("nilai y setelah pass by value : ", y)

    tambahReference(&y)
    fmt.Println("nilai y setelah pass by reference : ", y)
}
```

Penjelasan :

Program tersebut mendemonstrasikan perbedaan mendasar antara pengiriman argumen berdasarkan nilai (pass by value) dan berdasarkan referensi (pass by reference) menggunakan pointer di bahasa pemrograman Go. Pada fungsi tambahValue, variabel dikirim sebagai salinan sehingga perubahan nilai di dalam fungsi tersebut tidak memengaruhi variabel asli y di fungsi utama, yang menjelaskan mengapa nilai y tetap 5 setelah fungsi tersebut dipanggil. Sebaliknya, fungsi tambahReference menerima alamat memori dari variabel menggunakan operator *int dan memodifikasi nilai aslinya secara langsung melalui dereferensi (*x), sehingga setelah fungsi ini dijalankan, nilai variabel y di fungsi utama berubah secara permanen menjadi 15. Secara keseluruhan, kode ini mengajarkan bahwa penggunaan pointer

(simbol & untuk alamat dan * untuk nilai) sangat krusial ketika kita ingin sebuah fungsi memberikan dampak langsung pada data yang berada di luar cakupan fungsi tersebut.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "Week 4 - Modul 4/Guided/guided1/guid
ed-no1.go"
nilai y awal :  5
Nilai x didalam prosdur tambahValue (pass by value) :  15
nilai y setelah pass by value :  5
Nilai x didalam prosdur tambahReference (pass by refeence) :  15
nilai y setelah pass by reference :  15
PS D:\109082530025_MOH-Chandra-Wardana_>
```

4. Implementasi perhitungan dalam perhitungan transaksi belanja

```go
package main

import "fmt"

func hitungTotalBelanja(harga int, jumlah int){
    var total int
    total = harga * jumlah
    fmt.Println("total harga : ", total)

    hitungDiskon(total)
}

func hitungDiskon(total int){
    var diskon, hargaAkhir int
    diskon = total * 10/100 //contoh diskon 10%
    hargaAkhir = total - diskon

    fmt.Println("Diskon 10% : ", diskon)
    fmt.Println("harga akhir (setelah diskon): ", hargaAkhir)
}

func main(){
    var harga, jumlah int

    fmt.Print("masukkan harga : ")
    fmt.Scan(&harga)
    fmt.Print("masukkan jumlah : ")
    fmt.Scan(&jumlah)

    hitungTotalBelanja(harga, jumlah)
}
```

Penjelasan :
Program ini dibuat untuk mengatur proses perhitungan transaksi belanja supaya lebih rapi dengan membagi tugas ke beberapa prosedur. Di bagian main, program menerima input harga dan jumlah barang dari pengguna menggunakan fmt.Scan, lalu data tersebut dikirim ke prosedur hitungTotalBelanja untuk menghitung total awal (harga × jumlah). Setelah itu, perhitungan dilanjutkan ke prosedur hitungDiskon yang menghitung potongan harga sebesar 10% dan menentukan total akhir yang harus dibayar. Dengan cara ini, setiap bagian program punya tugas masing-masing, jadi alurnya lebih jelas, mudah dibaca, dan tidak menumpuk di satu tempat.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> go run "Week 4 - Modul 4/Guided/guided2/guid
ed-no2.go"
masukkan harga : 20000
masukkan jumlah : 5
total harga :  100000
Diskon 10% :  10000
harga akhir (setelah diskon):  90000
PS D:\109082530025_MOH-Chandra-Wardana_>
```

F. Unguided

3. Soal Unguided 3

   Deret Skiena

```go
package main

import "fmt"

func cetakDeret(n int) {
    for n != 1 {
        fmt.Printf("%d ", n)
        if n%2 == 0 {
            n = n / 2
        } else {
            n = 3*n + 1
        }
    }
    fmt.Println(1)
}

func main() {
    var bilangan int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&bilangan)
    cetakDeret(bilangan)
}
```

Penjelasan :

Program ini menerapkan algoritma deret bilangan yang dikenal sebagai masalah Collatz, di mana sebuah prosedur bernama cetakDeret menerima satu parameter bilangan bulat positif. Di dalam prosedur tersebut, digunakan perulangan while (dalam Go menggunakan for) yang akan terus berjalan selama nilai n belum mencapai angka 1, dengan logika percabangan untuk membagi n dengan dua jika bernilai genap atau mengalikannya dengan tiga dan menambah satu jika bernilai ganjil. Setiap angka yang dihasilkan dalam proses tersebut langsung dicetak dalam satu baris yang dipisahkan oleh spasi, sehingga ketika prosedur dipanggil di fungsi utama, ia akan menampilkan seluruh rangkaian deret dari angka awal hingga berakhir di angka satu.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_> cd "Week 4 - Modul 4/Unguided/unguided1"
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided1> go run u
nguided-no1.go
Masukkan bilangan: 22
22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided1> go run u
nguided-no1.go
Masukkan bilangan: 36
36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided1> go run u
nguided-no1.go
Masukkan bilangan: 60
60 30 15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided1>
```

4. Soal Unguided 4
   Program Bangun Datar

```go
package main

import (
    "fmt"
    "math"
)

func hitungPersegi(sisi int) {
    luas := sisi * sisi
    keliling := 4 * sisi
    fmt.Printf("Luas persegi: %d\n", luas)
    fmt.Printf("Keliling persegi: %d\n", keliling)
}

func hitungPersegiPanjang(panjang, lebar int) {
    luas := panjang * lebar
    keliling := 2 * (panjang + lebar)
    fmt.Printf("Luas persegi panjang: %d\n", luas)
    fmt.Printf("Keliling persegi panjang: %d\n", keliling)
}

func hitungLingkaran(jarijari float64) {
    luas := math.Pi * jarijari * jarijari
    keliling := 2 * math.Pi * jarijari
    fmt.Printf("Luas lingkaran: %f\n", luas)
    fmt.Printf("Keliling lingkaran: %f\n", keliling)
}

func main() {
    var pilihan int
    fmt.Println("PROGRAM BANGUN DATAR")
    fmt.Println("1. Hitung luas & keliling persegi")
    fmt.Println("2. Hitung luas & keliling persegi panjang")
    fmt.Println("3. Hitung luas & keliling lingkaran")
    fmt.Print("Pilihan: ")
    fmt.Scan(&pilihan)

    switch pilihan {
    case 1:
        var s int
        fmt.Print("Masukkan sisi: ")
        fmt.Scan(&s)
        hitungPersegi(s)
    case 2:
        var p, l int
        fmt.Print("Masukkan panjang: ")
        fmt.Scan(&p)
        fmt.Print("Masukkan lebar: ")
        fmt.Scan(&l)
        hitungPersegiPanjang(p, l)
    case 3:
        var r float64
        fmt.Print("Masukkan jari-jari: ")
        fmt.Scan(&r)
        hitungLingkaran(r)
    default:
        fmt.Println("Pilihan tidak valid.")
    }
}
```

Penjelasan :

Kode ini dirancang dengan memisahkan logika perhitungan setiap bangun datar ke dalam tiga prosedur berbeda, yaitu hitungPersegi, hitungPersegiPanjang, dan hitungLingkaran, yang masing-masing bertanggung jawab untuk menghitung sekaligus mencetak luas dan kelilingnya. Bagian utama program menampilkan menu pilihan kepada pengguna dan menggunakan struktur kendali switch-case untuk menentukan prosedur mana yang harus dijalankan berdasarkan input pilihan yang dimasukkan. Khusus untuk perhitungan lingkaran, digunakan tipe data float64 dan konstanta math.Pi agar hasil perhitungan luas serta keliling yang melibatkan bilangan desimal dapat ditampilkan dengan tingkat presisi yang akurat sesuai dengan ekspektasi output pada dokumen.

Output :

```
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided2> go run u
nguided-no2.go
PROGRAM BANGUN DATAR
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan: 1
Masukkan sisi: 1057
Luas persegi: 1117249
Keliling persegi: 4228
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided2> go run u
nguided-no2.go
PROGRAM BANGUN DATAR
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan: 2
Masukkan panjang: 106
Masukkan lebar: 88
Luas persegi panjang: 9328
Keliling persegi panjang: 388
PS D:\109082530025_MOH-Chandra-Wardana_\Week 4 - Modul 4\Unguided\unguided2> go run u
nguided-no2.go
PROGRAM BANGUN DATAR
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan: 3
Masukkan jari-jari: 13.87
Luas lingkaran: 604.369856
Keliling lingkaran: 87.147780
```

G. Kesimpulan

Modul 4 mengajarkan penggunaan prosedur, yaitu subprogram dalam bahasa Go yang tidak memiliki nilai kembalian (*return*) dan berfungsi untuk membungkus instruksi kompleks agar kode lebih rapi. Materi ini menekankan perbedaan antara parameter formal (deklarasi) dan aktual (pemanggilan) , serta penggunaan pointer (*) untuk *pass by reference* agar perubahan nilai di dalam prosedur tersimpan ke variabel utama. Penerapannya mencakup latihan pembuatan program matematika (faktorial, permutasi, kombinasi, dan fungsi komposisi) , pengolahan deret bilangan ganjil-genap (Skiena) , hingga pembuatan menu interaktif untuk menghitung luas dan keliling bangun datar menggunakan prosedur spesifik dan switch-case.